0.1 The Classical Synthesis

0.1.1 Monte Carlo Estimation

When $P(s'|s,a)$ and $r(s,a)$ are unknown, the obvious approach is to use Monte Carlo (MC) to approximate them. These methods estimate value functions from sampled *episodes* $(s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$. The realized *return* from state s_t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \tag{1}$$

First-visit MC prediction averages G_t over episodes for each state s, counting only its first occurrence per episode. Each first-visit return is an independent draw from the return distribution, so the sample mean converges almost surely to $V^\pi(s)$ by the strong law of large numbers (**?**). An incremental update is,

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$$

For MC control, we can estimate action-values $Q(s,a)$ by averaging first-visit returns from each state-action pair, then improve the policy greedily: $\pi(s) = argmax_a\, Q(s,a)$. Under exploring starts (every (s,a) pair begins an episode infinitely often), this alternation converges to Q^*. [1]

0.1.2 Sutton (1988)

Monte Carlo has two limitations. The agent must wait for episode termination to compute G_t, ruling out continuing tasks. And G_t is unbiased ($\mathbb{E}[G_t \mid S_t = s] = V^\pi(s)$) but high-variance, because it sums random rewards over the entire *trajectory*.

? proposed temporal difference (TD) learning to fix both problems. TD(0) replaces the full return G_t with a one-step target:

$$V(s_t) \leftarrow V(s_t) + \alpha\left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\right] \tag{2}$$

[1] **?** proved convergence even when the policy improves after every episode rather than waiting for complete evaluation. In practice, exploring starts is infeasible, so on-policy variants use ε-greedy exploration instead. These converge to Q^* provided the exploration schedule satisfies the greedy-in-the-limit with infinite exploration (GLIE) condition: every state-action pair is visited infinitely often, and $\varepsilon_t \to 0$ so the policy converges to greedy in the limit.

The target $r_{t+1} + \gamma V(s_{t+1})$ depends on one random reward and one random transition, so its variance is low. The cost is bias. The bootstrap target $V(s_{t+1})$ is the agent's current estimate, not the true value. This is "*bootstrapping*".[2] As V improves, the bias shrinks; the low variance persists regardless. Sutton demonstrated this tradeoff on a five-state random walk where TD(0) converged faster than Monte Carlo with less data.

The general TD(λ) update interpolates between these extremes through an eligibility trace.[3]

$$V(s_t) \leftarrow V(s_t) + \alpha\, \delta_t\, e_t(s) \tag{3}$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and $e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbb{1}\{s = s_t\}$ is the eligibility trace for state s. Setting $\lambda = 0$ yields TD(0); setting $\lambda = 1$ recovers Monte Carlo returns. Intermediate λ trades off variance against bias. [4]

0.1.3 Watkins (1989)

TD(0) learns value functions $V(s)$, but converting these to actions (the control problem) still requires knowing transition probabilities. Given $V(s')$ for all successor states, the agent needs to know which action leads to which successor.

?, formalizing Watkins's 1989 PhD thesis, provided the solution. Instead of learning $V(s')$ learn $Q(s, a)$ (the action value function or the "quality" of actions function) directly, the expected return from taking action a in state s and then behaving optimally. The optimal policy is then $\pi^*(s) = argmax_a\, Q^*(s, a)$, requiring no model to act. The Bellman optimality equation provides the fixed-point condition:

$$Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a\right] \tag{4}$$

Q-learning achieves this via the update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right] \tag{5}$$

[2]See Section **??** for the distinction between RL bootstrapping and Efron's resampling procedure.

[3]The eligibility trace records which states were recently visited, allowing credit assignment to propagate backward in time. States visited more recently receive stronger updates when the TD error is observed.

[4]**?** proved convergence of TD(λ) for general λ in the tabular case; **?** gave a unified stochastic approximation proof covering both TD and Q-learning; **?** extended the analysis to linear function approximation.

Q-learning converges to Q^* under standard regularity conditions (Section **??**). The maximization over a' makes Q-learning *off-policy*:[5] the update target uses the greedy action at the next state regardless of the action actually taken. Therefore the agent can follow an ε-greedy exploration strategy or even a fully random policy, while learning about the optimal policy directly.

0.1.4 Williams (1992)

Value-based methods learn an action-value function and derive a policy from it. **?** derived an alternative, policy gradients, that optimize the policy directly by gradient ascent on expected return

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\right] \tag{6}$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return from time t. The log-derivative trick allows the gradient to be estimated from sampled trajectories without differentiating through the environment dynamics.

Consider a robot arm that must apply a continuous torque $a \in \mathbb{R}$ to reach a target angle. Q-learning requires computing $\max_a Q(s, a)$ at every update, which becomes a nested optimization problem [6] when the action space is continuous. A policy gradient method sidesteps the issue. Parameterize the policy as a Gaussian $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s),\, \sigma^2_\theta(s))$,[7] sample an action, observe the return, and update θ by REINFORCE. Virtually all continuous-control results in reinforcement learning descend from the policy gradient framework for this reason.

0.1.5 Tesauro (1994)

?'s TD-Gammon demonstrated that temporal difference learning with neural network function approximation could achieve expert-level play in a domain with approximately 10^{20} legal positions[8].

[5] See Section **??** for the on-policy/off-policy distinction. Q-learning learns about the greedy policy $\pi^*(s) = \mathit{argmax}_a\, Q(s, a)$ while collecting data with an exploratory policy. This exploratory policy needs only to be sufficiently "exploratory" and admits a wide range of policies; including a fully random policy.

[6] In continuous action spaces, $\max_a Q(s, a)$ has no closed-form solution in general and must be solved numerically at every Bellman update. Discretizing a d-dimensional action space on a grid of m points per dimension costs $O(m^d)$ evaluations per update step.

[7] The Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ has density $(2\pi\sigma^2)^{-1/2} \exp(-(a-\mu)^2/2\sigma^2)$. Here $\mu_\theta(s)$ and $\sigma_\theta(s)$ are neural network outputs parameterizing the policy mean and standard deviation.

[8] The state $\mathbf{x}(s)$ was a 198-dimensional binary encoding of the raw board (four units per board point per player

Backgammon was far beyond tabular methods, yet TD-Gammon trained a feedforward neural network to estimate the probability of winning from any board position. A hidden layer of 80 sigmoid units fed a single sigmoid output.

$$\hat{V}(s) = \sigma\Big(\mathbf{w}^\top \sigma(W\mathbf{x}(s) + \mathbf{b}) + c\Big) \tag{7}$$

where σ is the logistic sigmoid, W is the input-to-hidden weight matrix, and $\mathbf{w}$ is the hidden-to-output weight vector. The network was trained by self-play using TD(λ) with $\lambda = 0.7$. After each move from position s_t to s_{t+1}, the weights $\boldsymbol{\theta}$[9] were updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\Big[\hat{V}(s_{t+1}) - \hat{V}(s_t)\Big]\,\mathbf{e}_t \tag{8}$$

where the eligibility trace $\mathbf{e}_t = \sum_{k=1}^{t} \lambda^{t-k} \nabla_{\boldsymbol{\theta}} \hat{V}(s_k)$ accumulates exponentially decayed gradients of past predictions.[10] At game's end, $\hat{V}(s_{t+1})$ is replaced by the outcome $z \in \{0, 1\}$.

A single neural network $\hat{V}$ serves as the evaluation function for both players. At each turn, the current player selects the legal move maximizing $\hat{V}(s')$ from its own perspective. As the network improves, it generates stronger play on both sides, producing harder training games that drive further improvement. The dice rolls ensure diverse board positions without requiring an explicit exploration mechanism.[11]

0.1.6 SARSA (1994)

Q-learning learns the optimal action-value function regardless of the policy generating experience. This off-policy property is useful but introduces complications when combined with function approximation. **?** introduced SARSA as an *on-policy*[12] alternative that learns the value of the policy actually being followed. The name derives from the quintuple

indicating checker counts, plus bar and borne-off counts). The output was a four-component vector estimating probabilities of each game outcome (White/Black $\times$ normal win/gammon), and the move maximizing expected outcome among all legal moves was selected at each step.

[9] $\boldsymbol{\theta}$ denotes the full vector of network weights and biases, generalizing the scalar θ used for policy parameters in earlier sections.

[10] In the neural network case, the eligibility trace $\mathbf{e}_t \in \mathbb{R}^{|\boldsymbol{\theta}|}$ is a vector accumulating exponentially-weighted gradients, extending the scalar state-based trace to parameter space.

[11] Version 2.1, trained on 1,500,000 games with 2-ply search, achieved near-parity with former world champion Bill Robertie and discovered novel positional strategies subsequently adopted by the human backgammon community.

[12] See Section **??** for the on-policy/off-policy distinction.

$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ used in each update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right] \tag{9}$$

The key difference from Q-learning is that SARSA bootstraps from the action a_{t+1} actually taken at the next state, rather than the greedy action $argmax_{a'} Q(s_{t+1}, a')$. This makes the algorithm on-policy, since the target depends on the behavior policy generating the data.

If the agent follows an ε-greedy policy, SARSA converges to $Q^{\varepsilon\text{-greedy}}$, not Q^*[13] policies and standard step-size conditions (Section **??**). This distinction matters when exploration is costly. Consider the cliff-walking problem, where an agent must traverse a gridworld with a cliff along one edge. The optimal path runs along the cliff edge (shortest route), but the ε-greedy policy occasionally falls off. Q-learning learns the optimal path because it evaluates the greedy policy; the agent falls off during learning but the Q-values reflect the optimal route. SARSA learns a safer path further from the cliff because it evaluates the actual exploratory policy; it accounts for the fact that exploration sometimes leads to catastrophic states.

0.1.7 Baird (1995)

? constructed a six-state star MDP demonstrating divergence of Q-learning with linear function approximation. The MDP has five outer states that all transition to a single inner state under the target policy. The off-policy behavior samples states uniformly. With linear function approximation, the weights grow without bound under repeated Q-learning updates. The source of instability is the interaction of three components, namely bootstrapping (updating from estimated values rather than observed returns), off-policy learning (training on data from a different policy than the target), and function approximation (representing the value function with a parameterized model). **?** later named this the deadly triad. Any two components can be combined safely; all three together permit divergence. The mechanism underlying this instability is analyzed in Section **??**.

[13] SARSA converges to Q^* under GLIE (greedy in the limit with infinite exploration)A GLIE schedule explores all state-action pairs infinitely often but converges to the greedy policy asymptotically. The standard ε-greedy policy with $\varepsilon_t \to 0$ is one such schedule.

This result explains the asymmetry between Tesauro's success and Baird's failure. TD-Gammon used bootstrapping and function approximation but was on-policy, with training data coming from the same self-play policy whose value was being estimated. Baird's counterexample used all three components and diverged. Baird also proposed a constructive solution, namely residual gradient algorithms that perform gradient descent on the mean-squared Bellman residual, guaranteeing convergence at the cost of a different fixed point.

0.1.8 Actor-Critic Methods (2000)

The idea of maintaining both a policy (*actor*) and a value function (*critic*) dates to **?**, who used a two-component system to solve the pole-balancing task. Actor-critic methods address the high variance of REINFORCE by replacing Monte Carlo returns with bootstrapped TD targets as the learning signal. The critic learns a value function $V(s)$ by TD updates.

$$V(s_t) \leftarrow V(s_t) + \alpha_c \, \delta_t, \quad \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \tag{10}$$

The actor updates the policy using the TD error as a sample of the advantage.

$$\theta \leftarrow \theta + \alpha_a \nabla_\theta \log \pi_\theta(a_t|s_t) \, \delta_t \tag{11}$$

The TD error δ_t estimates $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$, the advantage of action a_t over the average action.[14] Positive advantages indicate the action was better than expected; negative advantages indicate it was worse.

? provided the first convergence proof for actor-critic algorithms with function approximation, showing convergence to a stationary point under two-timescale learning rates ($\alpha_c \gg \alpha_a$) and a compatibility condition on the critic architecture (Section **??**).

0.1.9 Natural Policy Gradient (2001)

Standard gradient descent treats all parameter directions equally, but policy parameters define probability distributions whose natural geometry is not Euclidean. **?** introduced the natural policy gradient, which measures progress in distribution space rather than

[14] That δ_t is an unbiased estimate of the advantage follows from the policy gradient theorem (**?**).

parameter space. The update uses the Fisher information matrix $F(\theta)$:

$$F(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top\right] \tag{12}$$

The natural gradient is

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta) \tag{13}$$

This direction is invariant to reparameterization of the policy. In the tabular softmax[15] case, a single natural gradient step with unit step size recovers one step of exact policy iteration (Section **??**).

The computational bottleneck is inverting $F(\theta) \in \mathbb{R}^{d\times d}$. Practical implementations use conjugate gradient methods to solve $F(\theta)x = \nabla_\theta J(\theta)$ without forming F explicitly. This approach was later scaled to deep neural networks by TRPO and PPO.

0.1.10 Fitted Value Iteration and Fitted Q-Iteration (2005)

Tabular Q-learning maintains a separate entry for every state-action pair. When $|\mathcal{S}|$ is large or the state space is continuous, as in most economic applications, this is infeasible. *Fitted Q-Iteration* (FQI) (**?**) replaces the tabular update with a supervised regression step: given a batch of transitions, fit a function approximator to the Bellman targets.

Let $\mathcal{F}$ be a function class mapping $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Initialize $Q_0 \equiv 0$. At each iteration $k = 0, 1, \ldots, K-1$: (i) draw N transitions (s_i, a_i, r_i, s_i') from a generative model; (ii) construct regression targets $y_i^{(k)} = r_i + \gamma \max_{a'} Q_k(s_i', a')$; (iii) set $Q_{k+1} \leftarrow \arg\min_{f\in\mathcal{F}} \frac{1}{N}\sum_{i=1}^N \left(f(s_i, a_i) - y_i^{(k)}\right)^2$. The output is the greedy policy $\pi_K(s) = \arg\max_a Q_K(s, a)$.

The regression step replaces the exact Bellman application with a projection onto $\mathcal{F}$: $Q_{k+1} = \Pi_{\mathcal{F}} \mathcal{T} Q_k$, where $\mathcal{T}$ is the Bellman optimality operator and $\Pi_{\mathcal{F}}$ is the L^2-projection under the sample distribution. Fitted Value Iteration (FVI) (**?**) applies the same idea to the value function directly: $V_{k+1} = \Pi_{\mathcal{F}} \mathcal{T}^* V_k$, where $(\mathcal{T}^* V)(s) = \max_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')\}$.

With feature matrix $\Phi \in \mathbb{R}^{|\mathcal{S}|\times d}$ (rows $\phi(s)^\top$) and per-action weight vectors $\theta_a \in \mathbb{R}^d$, each FQI regression step for action a reduces to the normal equations:

$$\theta_a^{(k+1)} = \left(\Phi^\top \Phi\right)^{-1} \Phi^\top y_a^{(k)}, \tag{14}$$

[15] The softmax function maps a vector $\mathbf{z}$ to a probability distribution: $\mathrm{softmax}(z_i) = \exp(z_i)/\sum_j \exp(z_j)$. A softmax policy parameterizes action probabilities as $\pi_\theta(a|s) = \mathrm{softmax}(\theta_{s,a})$.

where $y_a^{(k)}(s) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} \phi(s')^\top \theta_{a'}^{(k)}$. Computation is $O(d^2|\mathcal{S}| + d^3)$ per action per iteration. The FVI update takes the same form with a single weight vector θ_V:

$$\theta_V^{(k+1)} = \left(\Phi^\top \Phi\right)^{-1} \Phi^\top V_{\text{target}}^{(k)}, \tag{15}$$

where $V_{\text{target}}^{(k)}(s) = \max_a \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a) \phi(s')^\top \theta_V^{(k)} \right\}$. The finite-sample error theory for these methods is developed in Section **??**.

0.2 The Deep Learning Era

0.2.1 Deep Q-Networks (2015)

? trained a single convolutional neural network[16] to play 49 Atari 2600 games directly from pixel inputs ($210 \times 160 \times 3$) and a scalar score, using no game-specific features.

The architecture processed four consecutive frames through three convolutional layers and a fully connected layer. The network $Q(s,a;\theta)$ was trained to minimize the squared temporal difference error

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta) \right)^2 \right] \tag{16}$$

Two innovations stabilized learning. *Experience replay* (**?**) stored transitions (s,a,r,s') in a buffer $\mathcal{D}$ and sampled uniformly for training, breaking the temporal correlation between consecutive updates. A *target network* used a frozen copy of parameters θ^-, updated periodically,[17] so the regression target does not shift with each gradient step.

DQN exceeded human-level performance on 29 of 49 games using a single architecture and hyperparameters. Games requiring long-horizon planning or sparse rewards, such as Montezuma's Revenge, remained difficult.

0.2.2 TRPO and PPO (2015, 2017)

Policy gradient methods suffer from a practical instability: a single large gradient step can move the policy into a region where performance collapses and recovery is slow.

[16] A convolutional neural network applies learned spatial filters to detect local patterns in grid-structured data, commonly used for image inputs.

[17] The buffer held 10^6 transitions; the target network θ^- was synchronized to θ every $C = 10{,}000$ steps.

? addressed this with Trust Region Policy Optimization (TRPO). TRPO solves the constrained optimization problem

$$\max_{\theta} \; L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta \tag{17}$$

where L is a surrogate objective based on the advantage function $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$.[18]

? proposed Proximal Policy Optimization (PPO) as a simpler alternative. PPO replaces the KL constraint with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \Big) \right] \tag{18}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio. The clipping removes the incentive for $r_t(\theta)$ to move outside the interval $[1-\varepsilon, 1+\varepsilon]$, penalizing large policy updates without explicitly computing a divergence measure.

PPO outperformed A2C, TRPO, and the cross-entropy method on continuous control benchmarks and achieved the highest average reward on 30 of 49 Atari games among the methods tested. PPO became the default policy optimization algorithm for large-scale RL applications, demonstrating that constraining the magnitude of policy updates is essential for stable optimization.

0.2.3 Soft Actor-Critic (2018)

? introduced Soft Actor-Critic (SAC), which adds entropy regularization to the actor-critic framework. The agent maximizes expected return plus an entropy bonus:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t)) \right) \right] \tag{19}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is a temperature parameter. The entropy bonus encourages exploration by penalizing deterministic policies. The optimal policy under this objective is softmax in the Q-values: $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, connecting to discrete choice models in econometrics.

SAC maintains two Q-networks (to reduce overestimation bias) and a policy network.

[18] The Kullback-Leibler divergence $D_{\text{KL}}(p\|q) = \sum_x p(x) \log(p(x)/q(x))$ measures statistical distance between distributions. The bar denotes expectation over states: $\bar{D}_{\text{KL}} = \mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s))]$.

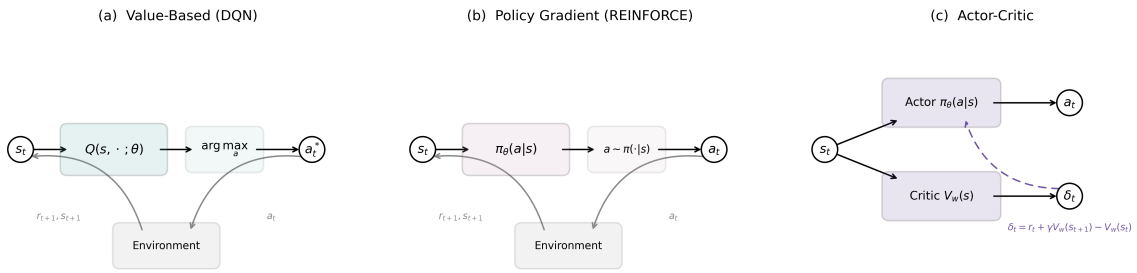

Figure 1: Architecture comparison of the three fundamental algorithm families. (a) DQN maps states to Q-values for all actions, selecting the argmax. (b) REINFORCE maps states to a probability distribution over actions, then samples. (c) Actor-Critic maintains separate policy and value networks; the critic's TD error δ_t provides a low-variance learning signal to the actor.

The soft Bellman operator for the critic is:

$$(T^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[V(s')\right], \quad V(s) = \mathbb{E}_{a\sim\pi}[Q(s,a) - \tau \log \pi(a|s)] \tag{20}$$

SAC is off-policy (using experience replay), handles continuous actions naturally, and achieves state-of-the-art sample efficiency on continuous control benchmarks. The entropy regularization provides automatic exploration without ε-greedy schedules.

0.2.4 AlphaGo Zero (2017)

The game of Go has approximately 10^{170} legal positions and a branching factor of roughly 250, far beyond the reach of brute-force search. Hand-crafted evaluation functions, which had succeeded in chess, failed here because positional concepts like influence, territory, and group viability are holistic and contextual. Monte Carlo tree search (MCTS) had achieved amateur-level play by using random simulations to estimate position values, but progress had stalled below professional strength. **?** broke through by combining supervised learning from 30 million human expert positions, reinforcement learning via self-play, and MCTS with learned value and policy networks; the resulting system defeated Lee Sedol four games to one in March 2016. A year later, **?** showed that none of the human data was necessary.

AlphaGo Zero uses a single convolutional neural network $f_\theta(s) = (\mathbf{p}, v)$ that takes a board position s and outputs both a policy vector $\mathbf{p}$ over legal moves and a scalar value v estimating the probability of winning. The input representation consists of 17 binary

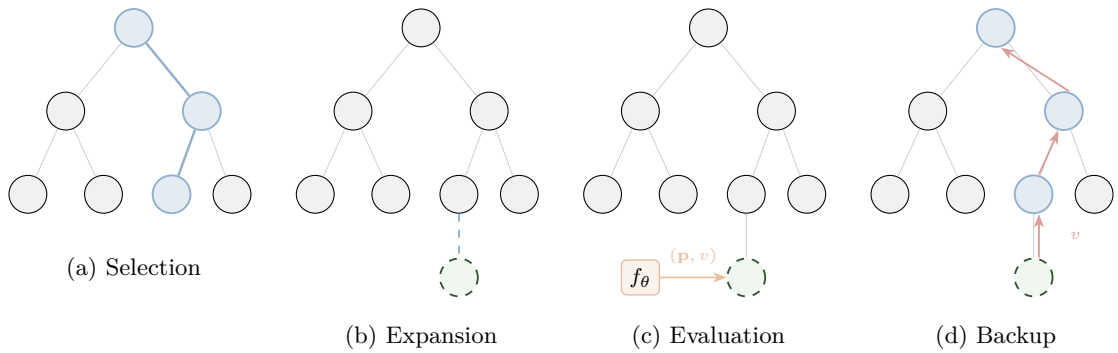

Figure 2: The four phases of a single MCTS simulation in AlphaGo Zero. (a) Selection traverses the tree from the root, choosing at each node the action maximizing a UCB-like score balancing exploitation (Q) and exploration (P/N). (b) Expansion adds a new leaf node when the traversal reaches an unexplored position. (c) The neural network f_θ evaluates the new position, producing move priors $\mathbf{p}$ and a value estimate v. (d) Backup propagates v along the traversed path, updating mean values $Q(s, a)$ and visit counts $N(s, a)$ at each edge.

planes on the 19×19 board encoding the raw game state without hand-crafted features.[19] The architecture uses residual blocks,[20] which allow training of very deep networks.

During play, each move is selected by running MCTS, which conducts 1,600 simulated games from the current position to estimate move quality. Each simulation proceeds in four phases, illustrated in Figure **??**. In the selection phase, the algorithm starts from the current position and traverses the partially built search tree by choosing at each node the action that maximizes $Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \sqrt{\sum_b N(s, b)} \,/\, (1 + N(s, a))$, where $Q(s, a)$ is the current average value of action a, $P(s, a)$ is the prior probability from the neural network, and $N(s, a)$ is the visit count.[21] In the expansion and evaluation phase, when the traversal reaches a position not yet in the tree, the neural network evaluates it in a single forward pass, producing a policy vector $\mathbf{p}$ and a value estimate v; the policy initializes prior probabilities $P(s', a) = p_a$ for each child edge. In the backup phase, the value v propagates back up the traversed path, incrementing each edge's visit count $N(s, a)$ and updating its mean value $Q(s, a)$. After all 1,600 simulations, the algorithm selects the move with the highest visit count at the root.

The training loop generates self-play games. At each board position s_t during a game,

[19] Eight planes for Black's stone positions over the last eight moves, eight for White's, and one indicating which color plays next. The history planes allow the network to detect ko situations and infer the trajectory of play.

[20] A residual block computes $\mathbf{x} + g(\mathbf{x})$ rather than just $g(\mathbf{x})$, where g is a learned transformation. The skip connection allows gradients to flow through very deep networks without vanishing. AlphaGo Zero's 40-block architecture has 79 parameterized layers.

[21] The constant c_{puct} controls exploration. Actions visited often have well-estimated Q values but a shrinking exploration bonus; rarely visited actions have uncertain values but a large bonus. This is a continuous analogue of the upper confidence bound (UCB) strategy from bandit theory.

the program runs MCTS to produce improved move probabilities $\boldsymbol{\pi}_t$, where $\pi_t(a) \propto N(s_t, a)^{1/\tau}$ and τ is a temperature parameter controlling exploration.[22] A move a_t is sampled from $\boldsymbol{\pi}_t$ and played. At game end, the outcome $z \in \{-1, +1\}$ is recorded. Each position becomes a training triple $(s_t, \boldsymbol{\pi}_t, z)$, and the network parameters are updated to minimize

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\theta\|^2 \tag{21}$$

where the first term is a value prediction loss, the second is a policy cross-entropy loss,[23] and the third is L_2 regularization.[24] The key mechanism is a virtuous cycle. MCTS serves as a policy improvement operator, since the search probabilities $\boldsymbol{\pi}$ are stronger than the raw network outputs $\mathbf{p}$. Training the network to match $\boldsymbol{\pi}$ distills the search improvements back into the network, and the improved network in turn produces better MCTS. After 72 hours of self-play on 4 TPUs, AlphaGo Zero surpassed all previous versions, including the one that defeated Lee Sedol, and discovered novel strategies not previously seen in human play.

Go was well-suited to this architecture. Its fixed 19×19 board maps naturally to convolutional networks, its perfect information and deterministic transitions make MCTS's tree structure exact, and the binary game outcome provides an unambiguous training signal. **?** interprets the architecture in econometric terms, where the policy network is a conditional choice probability (CCP) estimator, the value network is a conditional value function (CVF) estimator, and the system performs CCP estimation and forward simulation jointly, connecting to the approach of **?** in dynamic discrete choice.

[22] Early in the game ($t \leq 30$), $\tau = 1$ so moves are sampled proportionally to visit counts, encouraging diverse openings. Later, $\tau \to 0$ and the most-visited move is selected deterministically. Dirichlet noise is also added to root priors, $P(s, a) = (1 - \varepsilon)p_a + \varepsilon\eta_a$ with $\eta \sim \mathrm{Dir}(0.03)$, ensuring all legal moves can be explored despite strong network priors.

[23] The cross-entropy loss $-\boldsymbol{\pi}^\top \log \mathbf{p}$ measures how well the predicted distribution $\mathbf{p}$ matches the target $\boldsymbol{\pi}$; it equals zero when the distributions are identical.

[24] L_2 regularization penalizes the squared magnitude of parameters, $c\|\theta\|^2$, analogous to ridge regression in econometrics.